

6CCE3HAD Coursework Report

Contents

1 Introduction.....	1
2 General Design Methodology	2
2.1 High-Level System Overview.....	2
2.2 Block Diagram of the ALU Architecture	3
3 Detailed Design.....	4
3.1 Addition.....	4
3.2 Subtraction	5
3.3 Overflow Detection.....	6
3.4 Bitwise Logic Operations	7
3.5 Structural Multiplexer (8-to-1).....	7
3.6 Zero Flag Detection	9
3.7 Carry-out Flag Detection	9
4 Implementation	10
4.1 Vivado Project Structure	10
4.2 I/O Pin Mapping and Constraints File	10
4.3 RTL Schematic Analysis	12
5 Testing and Verification	13
5.1 Operation-by-Operation Verification	13
6 Alternative Implementation – Enable-Gated ALU.....	14
6.1 Overview of Alternative Design	14
6.2 Architectural Differences	15
6.3 Comparing ALU RTL Schematic.....	15
6.4 Equations and Logic	16
6.5 Testbench Verification.....	16

1 Introduction

The Arithmetic Logic Unit (ALU) is a key element in any processor, which is responsible for performing arithmetic and logical operations. In this coursework, we implement a 6-bit ALU using structural VHDL and deploy it onto a Basys3 FPGA board (Figure 1). As processors scale in complexity and speed, the need for reliable and efficient ALU designs becomes more important.

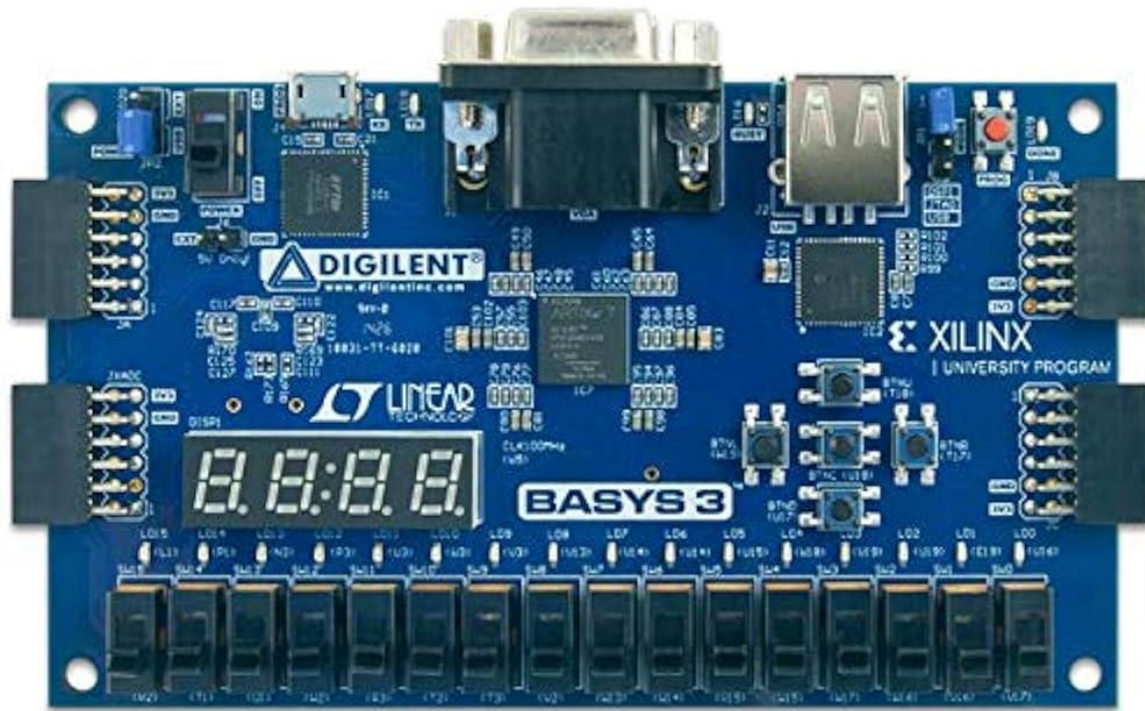


Figure 1 - Basys3 Artix-7 FPGA Board

The ALU processes two 6-bit inputs (A and B) and supports six operations: ADD, SUB, AND, OR, XOR, and NOR, which is chosen based on a 3-bit AluOp input. The output would be a 6-bit result along with three status flags: zero, carry out (Cout), and overflow (ofl).

The ALU must also be developed without any behavioural VHDL constructs such as: if, when and else and must not use any arithmetic library functions such as “+” or “-“. Instead, all operations must be implemented with gate-level logic, using components such as full adders, logic gates and multiplexers. The design will be tested in both simulation and on physical hardware, with inputs managed by switches and outputs shown through LEDs.

2 General Design Methodology

2.1 High-Level System Overview

The ALU is purely combinational system, which is made up of several reusable and logically defined components. At the top level, the ALU accepts three primary inputs and produces four outputs.

ALU Inputs:

- A[5:0] – 6-bit operand A
- B[5:0] – 6-bit operand B
- AluOp [2:0] – Choose operation

ALU Outputs:

- res[5:0] – Output result
- zero – Flag if res = 0
- cout – Carry out for ADD or SUB
- ofl – Overflow flag for ADD or SUB

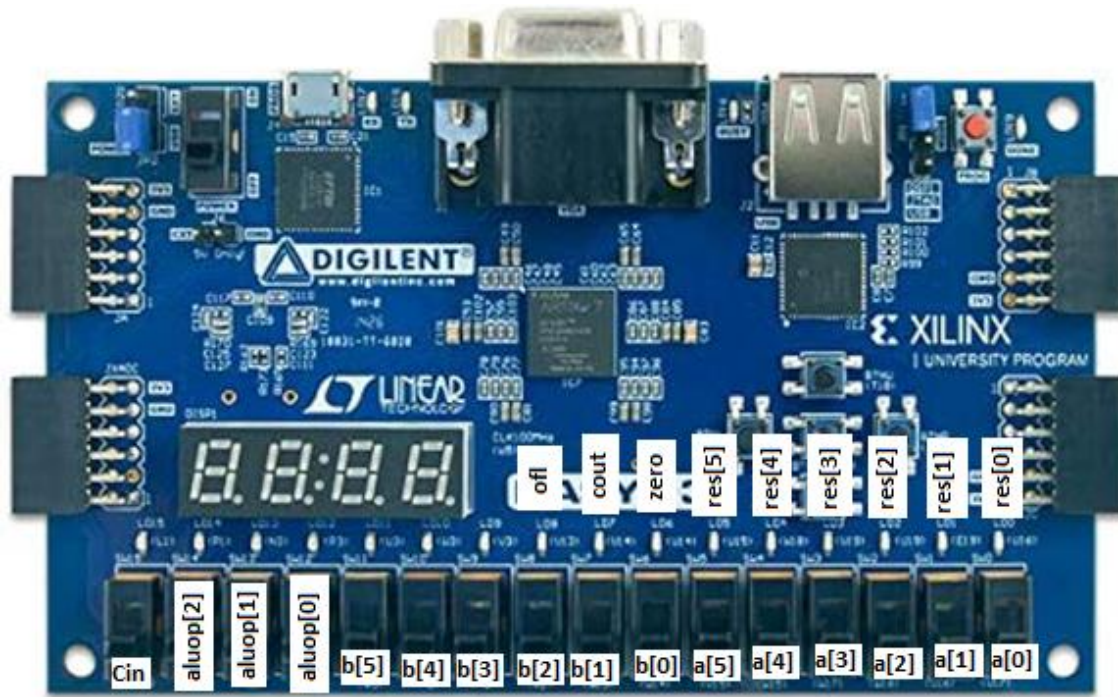


Figure 2 - Inputs Labelled on BASYS3 Board

Supported Operations:

AluOp	Mnemonic	Result	Description
000	add	$A + B$	Addition
010	sub	$A - B$	Subtraction
100	and	$A \text{ and } B$	Bitwise AND
101	or	$A \text{ or } B$	Bitwise OR
110	xor	$A \text{ xor } B$	Bitwise XOR
111	nor	$A \text{ nor } B$	Bitwise NOR
Others	-	Don't Care	-

Table 1 - Table of Operations for AluOp from Coursework Brief

2.2 Block Diagram of the ALU Architecture

We can break down the design of the ALU into smaller components to simplify each operation. Addition can be designed as 6x one-bit adder, or a 6-bit ripple carry adder. Subtraction can be designed with a two's complement with a 6-bit ripple carry adder. The other logic operations can be designed with a bitwise operation.

The ALU architecture is divided into these main blocks:

- 6-bit Ripple Carry Adder – to perform both ADD and SUB
- Two's Complement Unit – to negate B for SUB
- Bitwise Logic Blocks – AND, OR, XOR, and NOR operations
- Structural Multiplexers – Choose final results, cout, and ofl
- Flag Generators – Zero and overflow detection logic

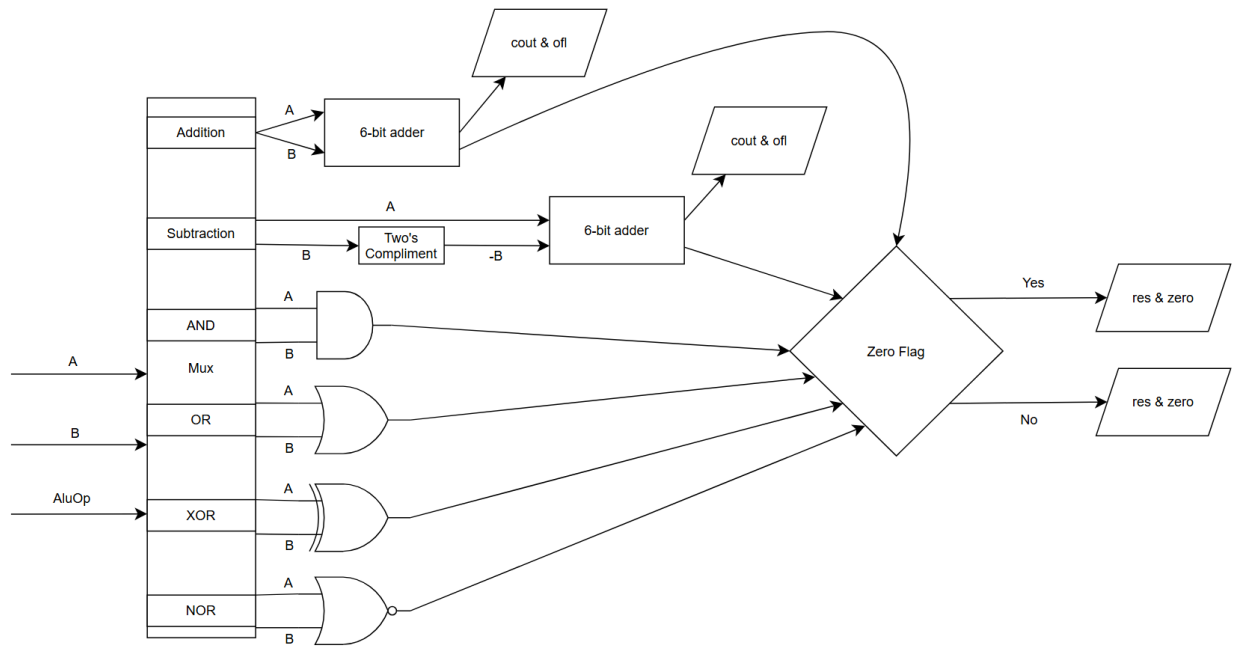


Figure 3 - ALU Block Diagram

3 Detailed Design

This section explains the inner workings of each component in the ALU.

3.1 Addition

The addition function is built using a 6-bit ripple carry adder made up on six 1-bit adder modules. Each full adder takes one bit from operand A, one bit from operand B, and a carry-in bit from the last stage. The carry-out (Cout) is passed to the next stage and the final carry-out becomes the Cout flag.

The equations used in each 1-bit full adder is:

$$Sum = A \oplus B \oplus Cin$$

$$Cout = (A \cdot B) \vee (Cin \cdot A) \vee (Cin \cdot B)$$

This 1-bit adder is implemented into VHDL as a separate file named full_adder.vhd and is shown in figure 4.

```
architecture gate_level of full_adder is
begin
    sum  <= a XOR b XOR Cin;
    cout <= (a AND b) OR (Cin AND a) OR (Cin AND b);
end gate_level;
```

Figure 4 - Full Adder Logic

In VHDL, rather than using six individual port map statements to map the adder, we used a generate loop to make the code more efficient shown in figure 5. The generate loop allows the code to be modifiable and allows it to scale from 6-bit to n-bits. It also mirrors real data path logic in FPGAs.

```

gen_add: for i in 0 to 5 generate
    adder_inst: full_adder port map(
        a      => a(i),
        b      => b(i),
        Cin    => carry_add_now(i),
        sum    => sumresult(i),
        cout   => carry_add_next(i)
    );
end generate;
Cout_add <= carry_add(6);

```

Figure 5 – Full Adder Generation Loop

The carry propagation across the 6-bit carry adder is managed by a structurally defined STD_LOGIC_VECTOR signal named carry_add, indexed from 0 to 6. In this implementation, carry_add(0) represents the initial carry-in for the least significant bit and carry_add(6) stores the last carry-out from the most significant bit. To avoid “+” or “-” library functions, we used aliases to construct a current carry and a next carry in figure 6.

```

alias carry_add_now  : STD_LOGIC_VECTOR(0 to 5) is carry_add(0 to 5);
alias carry_add_next : STD_LOGIC_VECTOR(0 to 5) is carry_add(1 to 6);

```

Figure 6 - Carry Propagation using Aliases

This structure allows each instance to clearly map its carry-in and carry-out signals. For each stage of i, carry_add_now provides the carry-in, and carry_add_next captures the carry out. This design makes sure that each stage of the ripple carry chain is connected properly. The generate loop shown in figure 5 also helps looping through the carries.

3.2 Subtraction

The subtraction function is implemented as $A + (\sim B + 1)$ using two’s complement. The inversion of B is done by a NOT bitwise function and the + 1 is injected using the adder’s initial carry-in set to 1. It also uses the same 6-bit adder module as addition, improving reusability.

The implementation starts with operand B being inverted using a bitwise NOT operation, which creates b_sub:

```

b_sub <= NOT b;

```

Figure 7 - Subtraction B Inversion

This signal holds the one’s compliment of B and is used as the second operand in the ripple-carry adder for subtraction. To complete the two’s complement operation, the initial carry-in to the adder is set to 1, which adds 1 to the inverted operand B, completing the two’s compliment.


```

-- set carry-in values
carry_add(0) <= '0';
carry_sub(0) <= '1';

```

Figure 8 - Setting Carry-in Values

Similarly to the addition logic, a generate loop is used to instantiate six 1-bit full adders for the subtraction data path shown in figure 9. Each full adder receives the i-th bit of operand A, the corresponding inverted bit from B, and the carry-in from the last stage. The sum is then stored in the signal subresult, which represents the 6-bit subtraction result.

```

gen_sub: for i in 0 to 5 generate
    sub_inst: full_adder port map(
        a    => a(i),
        b    => b_sub(i),
        Cin  => carry_sub_now(i),
        sum  => subresult(i),
        cout => carry_sub_next(i)
    );
end generate;
Cout_sub <= carry_sub(6);

```

Figure 9 - Subtraction Generation Loop

The subtraction function mirrors the carry structure of the addition path, using a STD_LOGIC_VECTOR signal carry_sub(0 to 6) to store carry values.

```

alias carry_sub_now  : STD_LOGIC_VECTOR(0 to 5) is carry_sub(0 to 5);
alias carry_sub_next : STD_LOGIC_VECTOR(0 to 5) is carry_sub(1 to 6);

```

Figure 10 - Subtraction Carry Propagation using Aliases

3.3 Overflow Detection

The ALU is required to signal whether an overflow has occurred during operations such as ADD or SUB. In digital systems, overflow happens when the result of an operation is larger than the representable range of the signed number format.

The overflow (ofl_add) is determined by comparing the sign bits of A, B and the result. If two positive numbers produce a negative result, or two negative numbers produce a positive result, overflow has occurred. This logic is implemented with:

$$Overflow = (A(5) \oplus B(5)) \cdot (A(5) \oplus Result(5))$$

The first term $A(5) \oplus B(5)$, checks whether A and B have the same sign. If it is false, overflow cannot happen. The second term $A(5) \oplus Result(5)$, checks whether the result has a different sign from the inputs. If both conditions are true, then overflow is detected. This logic is very efficient and compact, only needing XOR and AND gates, which is ideal for the implementation in the code.

The overflow is detected separately for addition and subtraction as each path uses different operand combinations. Since the overflow signals are calculated in parallel for both ADD and SUB operations, the correct one must be selected based on the current operation. This is handled structurally using AND and OR gates. The overflow logic is shown in figure 11.

```
Ofl_add <= (a(5) XOR b(5)) AND (a(5) XOR sumresult(5));
Ofl_sub <= (a(5) XOR b_sub(5)) AND (a(5) XOR subresult(5));

oFl <= (Ofl_add AND (NOT aluop(2) AND NOT aluop(1) AND NOT aluop(0))) OR
      (Ofl_sub AND (NOT aluop(2) AND aluop(1) AND NOT aluop(0)));
```

Figure 11 - Overflow Logic

The overflow selector uses gate-level decoding of AluOp to choose which the current operation is. Subsequently only the overflow from the active operation is passed to the output.

3.4 Bitwise Logic Operations

In addition to arithmetic functions, the ALU must perform four standard bitwise logical operations. The bitwise operations are carried out bit by bit for A and B. AND, OR, XOR, and NOR logic gates are applied to each bit and each operation has its own output signal vector which is fed into the multiplexer.

```
-- logic operations
andresult <= a AND b;
orresult  <= a OR b;
xorresult <= a XOR b;
norresult <= NOT (a OR b);
```

Figure 12 - Logic Operation Logic

These results are calculated in parallel, which means all the six bits of each operation are calculated simultaneously. There is no carry propagation to worry about, which makes these blocks significantly faster and more compact than the arithmetic paths.

3.5 Structural Multiplexer (8-to-1)

The operation selection is handled using gate-based logic to bypass the behavioural constraints. This section is based on AluOp and a selection signal is generated for each operation. The multiplexer first decodes the 3-bit AluOp into distinct operation selectors. Each unique combination of the AluOp bits matches an active selector line (such as sel_add, sel_sub, or sel_and), while all others are deactivated. The selector lines are constructed using AND and NOT gates. This ensures only one operation path is active at a time and all other operations are gated off.

The 3-bit selector is decoded into eight signals (sel(0) to sel(7)) using AND and NOT gates:

```

-- inverted selection lines
sel0_n <= NOT S(0);
sel1_n <= NOT S(1);
sel2_n <= NOT S(2);

-- selection logic
sel(0) <= sel2_n AND sel1_n AND sel0_n; -- 000
sel(1) <= sel2_n AND sel1_n AND S(0);   -- 001
sel(2) <= sel2_n AND S(1) AND sel0_n;   -- 010
sel(3) <= sel2_n AND S(1) AND S(0);     -- 011
sel(4) <= S(2) AND sel1_n AND sel0_n;   -- 100
sel(5) <= S(2) AND sel1_n AND S(0);     -- 101
sel(6) <= S(2) AND S(1) AND sel0_n;     -- 110
sel(7) <= S(2) AND S(1) AND S(0);       -- 111

```

Figure 13 - Selection Logic

This logic ensures that only one selector line is active at any given time, depending on the value of AluOp. The next part is how to calculate the output. Each output bit is calculated using a generate block that applies a bitwise AND between each input's bit and its selector, followed by an OR of all results shown in figure 14. Using the AND-OR logic allows us to bypass the behavioural restrictions and break down the original method with gate-level logic.

```

-- bitwise AND and OR logic to select correct input
gen_output: for i in 0 to 5 generate
    output(i) <=
        (I0(i) AND sel(0)) OR
        (I1(i) AND sel(1)) OR
        (I2(i) AND sel(2)) OR
        (I3(i) AND sel(3)) OR
        (I4(i) AND sel(4)) OR
        (I5(i) AND sel(5)) OR
        (I6(i) AND sel(6)) OR
        (I7(i) AND sel(7));
end generate;

```

Figure 14 - Output Computation Using AND-OR Logic

The MUX is mapped in the ALU in figure 15. Only six operations are currently used and the two unused slots I1 and I3 are filled with zero vectors. This allows for the ALU to have more space for more operations using the unused slots. The output of the MUX is assigned to the alu_result signal, which is mapped to the final output res.


```

-- MUX for result
MUX8: mux8to1 port map(
    I0 => sumresult,
    I1 => (others => '0'),
    I2 => subresult,
    I3 => (others => '0'),
    I4 => andresult,
    I5 => orresult,
    I6 => xorresult,
    I7 => norresult,
    S  => aluop,
    output => alu_result
);

```

Figure 15 - MUX Mapping

This MUX implementation method avoids behavioural control structures as no if, case, or when-else statements are used. The code is also all gate level, which means AND and OR operations are mapped directly to gates on the FPGA. The design of the MUX is scalable, which allows for future improvements to the size of the inputs with minimal effort. Finally, having the multiplexer in its own entity enhances the modularity and clarity of the project, which improves readability and allows for easier maintenance when testing.

3.6 Zero Flag Detection

In the ALU, the zero flag is a status signal that is used to show whether the result of an operation is zero. It is implemented using a structural OR-tree followed by an inverter, which makes it follow the gate-level logic structure. The idea behind the zero detection is if every bit of the 6-bit result vector is zero, then the combined output of the OR of all the bits will also be zero. By inverting the output, we produce a zero flag that is “1” only when all the bits in the result are zero.

```

-- zero flag
zero <= NOT (alu_result(0) OR alu_result(1) OR alu_result(2) OR alu_result(3) OR alu_result(4) OR alu_result(5));

```

Figure 16 - Zero Flag Logic

The code in figure 16 shows the implementation of the zero flag code, where alu_result(i) is the selected result from the 8-to-1 multiplexer and the zero is a 1-bit signal declared in the ALU entity output port.

3.7 Carry-out Flag Detection

The carry-out (Cout) flag is used to signal that an addition or subtraction operation has produced a carry beyond the most significant bit. For unsigned arithmetic functions, the flag is essentially detecting overflows, but in the context of the 6-bit ALU, the cout output reflects the carry-out bit from the most significant bit during addition or subtraction. Logical operations do not produce or use a carry-out flag.

```
-- carry out for add/sub
cout <= (Cout_add AND (NOT aluop(2) AND NOT aluop(1) AND NOT aluop(0))) OR
        (Cout_sub AND (NOT aluop(2) AND aluop(1) AND NOT aluop(0)));
```

Figure 17 - Carry Out Logic

4 Implementation

The ALU described in section 2 and 3 was implemented, synthesised, and tested using Xilinx Vivado on the Diligent BASYS3 board, built on the Xilinx Artix-7 XC7A35T FPGA. This section describes the steps taken to integrate all VHDL components, map them to the physical hardware, and confirm the synthesis using Vivado's built-in tools.

4.1 Vivado Project Structure

The ALU design was divided into modular VHDL files, where each file represents a component in the data path. This structure allows for easy debugging and keeps the design clear as shown in figure 18. The ALU.vhd is the top-level entity that combines all components and maps everything to the BASYS3 FPGA. The full_adder.vhd defines the 1-bit full adder. The mux8to1.vhd defines the multiplexer. Finally the constraints file is used to map logical ports to the FPGA I/O pins

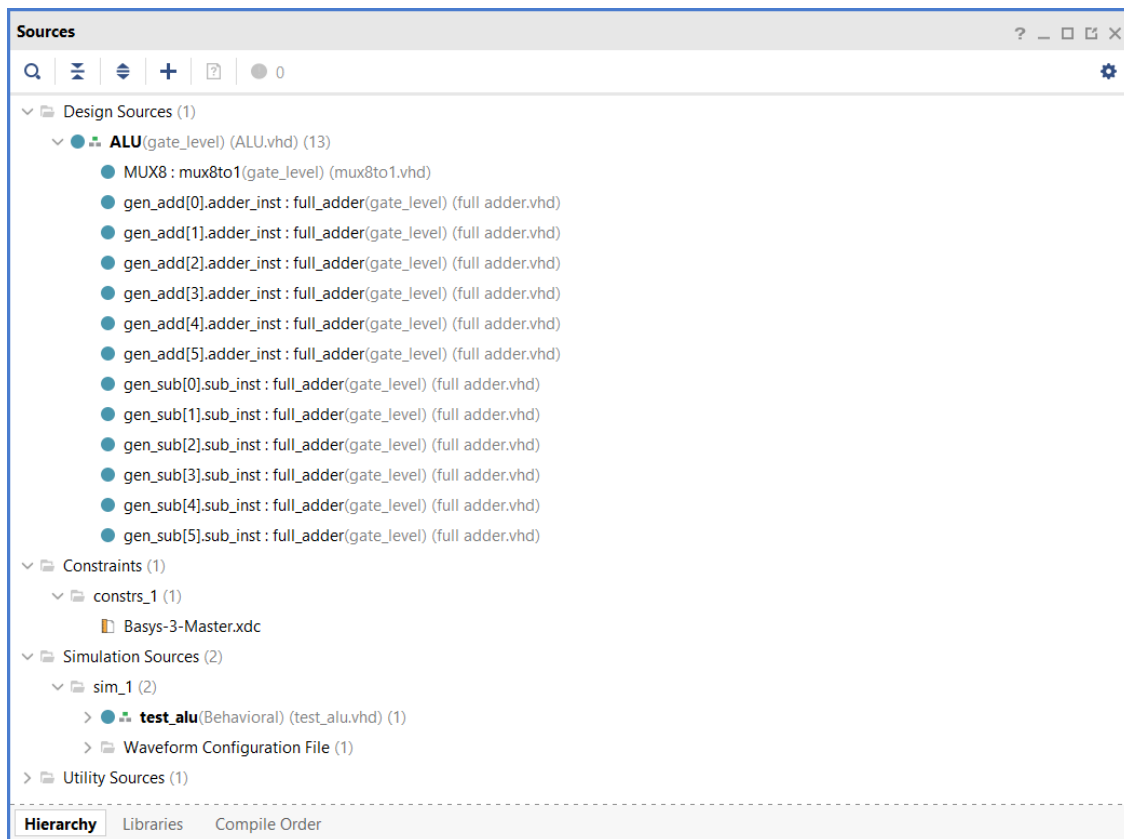


Figure 18 - Design Sources, Constraints File and Simulation Sources

4.2 I/O Pin Mapping and Constraints File

To interface the ALU with physical switches and LEDs on the BASYS3 board, we use a constraints file (.xdc) to bind the VHDL ports to specific FPGA pins. Table 2 shows all the ports we need in the

ALU with the ports we need on the BASYS3 board. Figure 19 also shows our constraints file we use to map the ALU onto the BASYS3 board.

a[0]	V17
a[1]	V16
a[2]	W16
a[3]	W17
a[4]	W15
a[5]	V15
b[0]	W14
b[1]	W13
b[2]	V2
b[3]	T3
b[4]	T2
b[5]	R3
aluop[0]	W2
aluop[1]	U1
aluop[2]	T1
Cin	R2
res[0]	U16
res[0]	E19
res[0]	U19
res[0]	V19
res[0]	W18
res[0]	U15
zero	U14
cout	V14
ofl	V13

Table 2 - Port Assignment

```

## Switches
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {a[0]}]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {a[1]}]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {a[2]}]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {a[3]}]
set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports {a[4]}]
set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports {a[5]}]
set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports {b[0]}]
set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports {b[1]}]
set_property -dict { PACKAGE_PIN V2 IOSTANDARD LVCMOS33 } [get_ports {b[2]}]
set_property -dict { PACKAGE_PIN T3 IOSTANDARD LVCMOS33 } [get_ports {b[3]}]
set_property -dict { PACKAGE_PIN T2 IOSTANDARD LVCMOS33 } [get_ports {b[4]}]
set_property -dict { PACKAGE_PIN R3 IOSTANDARD LVCMOS33 } [get_ports {b[5]}]
set_property -dict { PACKAGE_PIN W2 IOSTANDARD LVCMOS33 } [get_ports {aluop[0]}]
set_property -dict { PACKAGE_PIN U1 IOSTANDARD LVCMOS33 } [get_ports {aluop[1]}]
set_property -dict { PACKAGE_PIN T1 IOSTANDARD LVCMOS33 } [get_ports {aluop[2]}]
set_property -dict { PACKAGE_PIN R2 IOSTANDARD LVCMOS33 } [get_ports {Cin}]

## LEDs
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports {res[0]}]
set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports {res[1]}]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports {res[2]}]
set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports {res[3]}]
set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports {res[4]}]
set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports {res[5]}]
set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports {zero}]
set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVCMOS33 } [get_ports {cout}]
set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports {ofl}]

```

Figure 19 - Constraints File

4.3 RTL Schematic Analysis

Vivado can generate an RTL schematic to visually confirm how the components connect and its structure. From Figure 20, we can make these observations

- Each instance of full adder appears as a separate block
- The mux8to1 is displayed as its own block and has a structured logic tree with AND-OR networks)
- Internal signals like carry_add, b_sub, and alu_result are traceable across components.
- Flag logic such as zero, cout, ofl, are clearly displayed as combinational logic trees that attach to their results.

This shows that the design is fully structural and there is no behavioural logic shown during the synthesis of my code.

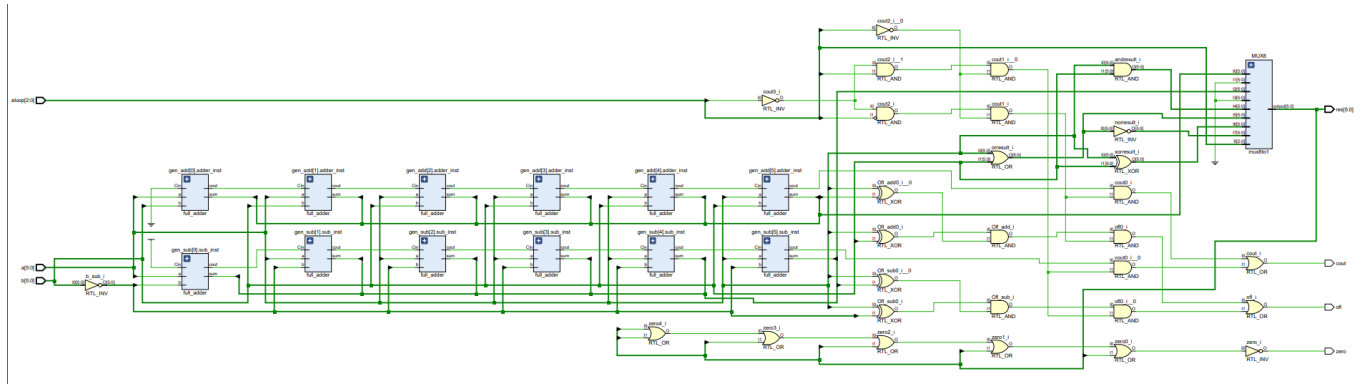


Figure 20 - RTL Schematic

5 Testing and Verification

Testing and verification are essential steps in digital design. They make sure that the implemented circuit synthesises properly and behaves as intended across all operations and edge cases.

5.1 Operation-by-Operation Verification

The testbench provided shown in Figure 21, shows the edge cases and operations we need to carry out in our ALU. Figure 22 shows my ALU's results from running the testbench.

```
| -- Please Note: These are just placeholder values given as reference.
| -- Actual values may be different during evaluation of the assignment.
| process begin
|     wait for global_wait;
|     a <= "110101"; b <= "100101"; aluop <= "000"; wait for 20 ns;
|     a <= "010101"; b <= "011111"; aluop <= "000"; wait for 20 ns;
|     a <= "100001"; b <= "111110"; aluop <= "010"; wait for 20 ns;
|     a <= "110101"; b <= "111110"; aluop <= "010"; wait for 20 ns;
|     a <= "010010"; b <= "101000"; aluop <= "111"; wait for 20 ns;
|     a <= "111111"; b <= "001100"; aluop <= "101"; wait for 20 ns;
|     a <= "010010"; b <= "101000"; aluop <= "110"; wait for 20 ns;
|     a <= "111111"; b <= "001100"; aluop <= "100"; wait for 20 ns;
|     wait;
| end process;
```

Figure 21 - Testbench Operations



Figure 22 - Testbench Results

a	b	aluop	Expected res	res	zero	cout	ofl
110101	100101	000	011010	011010	0	1	0
010101	011111	000	110100	110100	0	0	0
100001	111110	010	100011	100011	0	0	0
110101	111110	010	110111	110111	0	0	0
010010	101000	111	000101	000101	0	0	0
111111	001100	101	111111	111111	0	0	0
010010	101000	110	111010	111010	0	0	0
111111	001100	100	001100	001100	0	0	0

Table 3 - Expected Results and Results Table

Table 3 shows a compiled table of the expected results from the testbench as well as the results from my testbench output. It shows that all my ALU's results match the expected results which is a successful testbench simulation.

6 Alternative Implementation – Enable-Gated ALU

6.1 Overview of Alternative Design

In the original implementation of the ALU, all results were calculated in parallel and then routed through an 8-to-1 multiplexer to select the result based on the 3-bit aluop signal. While this approach is clean and modular, the MUX becomes the centre of all the control logic, which makes the design reliant on conditional selection structures.

As an alternative, the bonus implementation replaces the central mux with an enable-gated parallel logic architecture. This structure makes it so that all six operations are computed in parallel, but the result is now gated directly to each result path with an enable signal derived from the aluop control. The final output is constructed as an OR-combinaition of the gated signals.

6.2 Architectural Differences

Feature	Original Design	Alternative Design
Selection Mechanism	8-to-1 Multiplexer	Enable-gated logic blocks
Control signal decoding	Inside MUX	External enable signals
Output construction	Single vector selected via MUX	Parallel blocks gated and ORed
Result logic port	Res outputted directly	Internal signal used named <code>res_signal</code>
Flexibility for expansion	Limited by MUX inputs	Easy to add more operations
Bonus criteria?	Single-structure selection	Different logic level design

Table 4 - Differences Between Original and Alternative Design

6.3 Comparing ALU RTL Schematic

A clear structural difference between the original ALU design and the alternative design can be observed in the Vivado RTL schematics. The original design routes all operational outputs into a 8-to-1 multiplexer. This multiplexer is controlled by the `aluop` signal and it selects one operational result for the output. While effective and logically clean, this approach uses a single point of control and makes the design depend on that single point for all its conditional logic.

On the other hand, the alternative design eliminates the MUX entirely. Instead, each operation's result is gated by an enable signal derived from the `aluop` input. All operation blocks are active in parallel, but the only one with its enabled signal active contributes to its output. This produces a broader, more distributed logic structure where selection is performed at the signal level through enable gating and OR combinations. The schematic in Figure 23 clearly shows a wider result path which shows a different architectural approach than the original design.

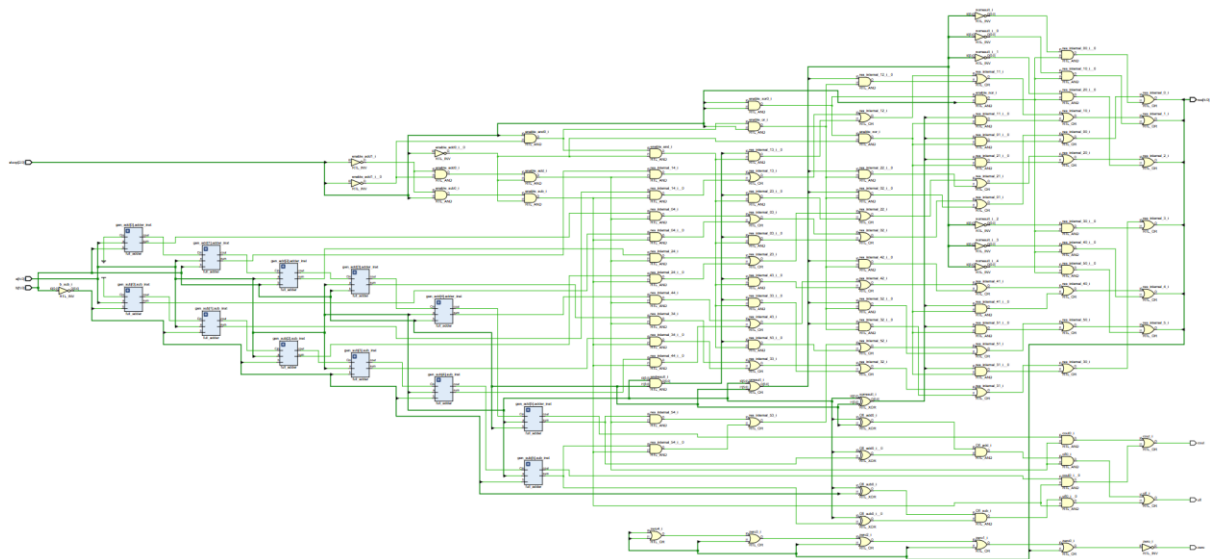


Figure 23 - Alternative Design RTL Schematic

6.4 Equations and Logic

Enable signals are derived from the aluop control using Boolean logic:

```
-- Enable decoding
enable_add <= NOT aluop(2) AND NOT aluop(1) AND NOT aluop(0); -- 000
enable_sub <= NOT aluop(2) AND aluop(1) AND NOT aluop(0);      -- 010
enable_and <= aluop(2) AND NOT aluop(1) AND NOT aluop(0);     -- 100
enable_or  <= aluop(2) AND NOT aluop(1) AND aluop(0);         -- 101
enable_xor <= aluop(2) AND aluop(1) AND NOT aluop(0);         -- 110
enable_nor <= aluop(2) AND aluop(1) AND aluop(0);             -- 111
```

Figure 24 - Decoding Logic

Each bit of the result vector is constructed from an OR of gated blocks:

```
-- output with enable gating
gen_res: for i in 0 to 5 generate
begin
    res_internal(i) <=
        (sumresult(i) AND enable_add) OR
        (subresult(i) AND enable_sub) OR
        (andresult(i) AND enable_and) OR
        (orresult(i) AND enable_or) OR
        (xorresult(i) AND enable_xor) OR
        (norresult(i) AND enable_nor);
end generate;
```

Figure 25 - Enable gating output

The result is assigned using this logic:

```
-- assign the internal result to the output port
res <= res_internal;
```

Figure 26 - Result Assigning Logic

6.5 Testbench Verification

Figure 27 shows that all results are the same as the working test bench previously, confirming our ALU design is working.

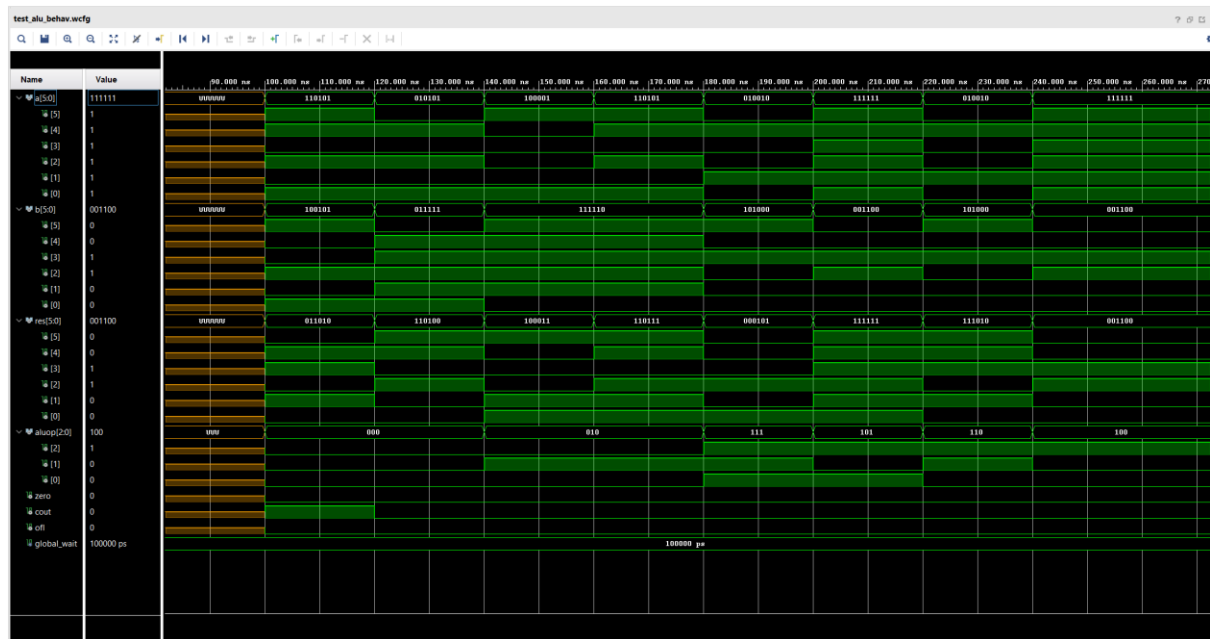


Figure 27 - Testbench for Alternative Design